

OrderDeltaBench: Evaluating State-Preserving Cart Edits in Large Language Model Ordering Agents

Yin Li*

University of Birmingham, Edgbaston, Birmingham B15 2TT, United Kingdom
kx1474@student.bham.ac.uk

May 2026

Abstract

Structured-output interfaces make language-model tool calls parseable, but transaction agents must also update persistent state correctly. A restaurant ordering agent rarely receives only one clean order: users add items, remove one of several similar lines, change modifiers, reverse earlier decisions, and introduce allergy or dietary constraints after a cart already exists. We introduce ORDERDELTABENCH, a deterministic benchmark for stateful cart edits in which each model must transform a current order into a safe updated transaction. The benchmark compares two executable interface designs: full order rewriting and minimal stable-line operations. Across 5,040 Nebius calls to seven Nebius-accessible open-weight models, all evaluated with strict structured-output schemas, line-keyed patches reduce state corruption when identity and scope are the bottleneck, but do not solve semantic safety. Aggregated across models, line patch raises exact semantic success from 58.1% to 62.0%, reduces unintended cart drift from 6.3% to 3.6%, and reduces wrong-line identity errors from 2.5% to 0.8%. The effect is model-dependent: Qwen3-32B improves from 64.7% to 80.8%, and Llama-3.1-8B improves from 25.3% to 48.1%, while Qwen3-235B-A22B, Llama-3.3-70B, Qwen3-30B-A3B, and Nemotron-Ultra-253B slightly favor rewrite on exact semantic success. The hardest categories are not JSON formatting tasks: newly conflicting constraints, unavailable edits, stale references, and modifier reversals remain unreliable despite valid schemas. Stable line identifiers reduce specific state-update errors, but transaction agents still need domain verification and fail-closed execution around safety and catalog rules.

Keywords: large language models; tool use; structured output; stateful agents; transaction safety; JSON Patch; empirical software engineering

1 Introduction

LLM agents are increasingly used as software components that translate natural language into executable API calls. In transactional systems, the risky output is often not malformed text. It is a well-formed object that changes the wrong state. A user may say “remove the plain vegan bowl”, “put the cheese back but keep no onion”, or “I forgot to say I have a peanut allergy” after the cart already contains multiple lines. A safe system must identify the intended line, preserve unrelated lines, update constraints, and refuse or clarify when the requested edit would make the transaction unsafe.

*Corresponding author. Email: kx1474@student.bham.ac.uk

This paper studies that state-update layer. Prior work on structured outputs and function calling has improved syntactic adherence to schemas [8, 4, 9, 13]. Agent benchmarks have also moved toward interactive, stateful tasks [14, 5, 6]. However, a deployment engineer still faces a narrower interface-design question: when an LLM updates a persistent transaction, should it regenerate the whole resource or return a minimal patch over stable object identifiers?

ORDERDELTABENCH isolates this question in restaurant ordering. The domain is small enough to verify exactly, but rich enough to contain common production failure modes: duplicate lines, scoped modifiers, quantity deltas, reversal of previous modifiers, unavailable catalog entries, stale references to absent cart lines, and user constraints that newly conflict with the current cart. The benchmark compares:

- **Full-order rewrite:** the model receives the current cart and returns the entire updated order.
- **Line-ID patch:** the model receives stable cart line IDs and returns typed operations such as `add_item`, `remove_line`, `update_line`, and `set_constraints`.

The study contributes:

- a deterministic 360-case benchmark for stateful transactional edits;
- two strict JSON Schema interface contracts for the same task;
- an executable verifier that applies patches, normalizes final orders, and scores semantic success, unsafe state changes, unintended drift, and line-identity errors;
- a paired empirical comparison across seven open models and 5,040 provider calls.

2 Related Work

Structured output and function calling. JSON Schema and provider structured-output modes define the structural contract between model and software system [4, 8]. BFCL evaluates whether models can select functions and produce suitable arguments [9], while Gorilla exposed hallucinated APIs as a core limitation of tool use [10]. StructEval broadens the view of structured generation across renderable and non-renderable formats [13]. ORDERDELTABENCH assumes strict schemas are useful, but measures the next layer: whether the resulting object correctly updates a transactional state.

Stateful tool-use benchmarks. τ -bench and ToolSandbox evaluate agents in interactive domains with state and user interaction [14, 5]. WebMall and ShoppingComp focus on shopping-agent retrieval, comparison, and decision quality [12, 11]. EnterpriseOps-Gym evaluates long-horizon enterprise tool use with persistent databases and policy constraints [6]. SWE-bench-Live illustrates the same methodological direction in software engineering: scalable, executable, continuously refreshed evaluation with explicit environment validation [15]. These benchmarks are broader than ORDERDELTABENCH. Our benchmark is deliberately surgical: it fixes the domain and prompt context so that interface choice and state-update semantics can be measured case by case.

Patch-based editing. JSON Patch defines a sequence of operations for partial updates to JSON documents [2]. Recent LLM work on JSON editing argues that patch generation can reduce token cost, and that stable keys help avoid array-index errors [3]. ORDERDELTABENCH adapts that intuition to transactional agents: the key question is not only efficient editing, but whether line-keyed operations reduce wrong-line edits, cart drift, and unsafe state changes.

Reporting empirical LLM studies. Because model APIs, model snapshots, and provider behavior change over time, empirical LLM studies need explicit model identifiers, prompts, logs, configuration, metrics, and limitations [1]. The artifact records exact provider model IDs, raw responses, usage fields, latency, prompts, generated cases, and verifier outputs.

3 Benchmark Design

3.1 Domain Contract

ORDERDELTABENCH reuses a compact restaurant menu with seven SKUs: classic burger, vegan bowl, chicken wrap, satay noodles, margherita pizza, fries, and cola. Each SKU has allowed sizes, default modifiers, removable modifiers, addable modifiers, allergens, and dietary tags. A cart line contains a stable `line_id`, SKU, quantity, size, additions, removals, and free-form special instructions. A cart also contains stated allergen and dietary constraints.

Each benchmark case has a current order, a short edit history, a user edit utterance, an expected status, an expected final order, and audit metadata marking target and unchanged line IDs. The edit history makes the prompt more like a stateful transaction, while the current order remains the authoritative state for scoring. The expected status is `accepted`, `needs_clarification`, or `rejected_safety`. When an edit is not accepted, the verifier expects fail-closed behavior: the model should not execute the requested cart edit.

3.2 Categories

The 360 cases are evenly distributed across 12 categories:

1. adding a valid item to an existing cart;
2. removing one line among similar duplicate lines;
3. increasing a line quantity;
4. decreasing a line quantity;
5. adding a modifier to only one referenced line;
6. reversing one previous modifier decision while keeping another;
7. changing the size of one duplicate line;
8. replacing an item while preserving sides or drinks;
9. adding a safe allergen or dietary constraint;
10. introducing a constraint that conflicts with the current cart;
11. requesting an unavailable item or modifier;
12. referring to a stale line that is not in the current cart.

Each category contains 30 cases generated deterministically from eight hand-written semantic templates plus lexical prefix and suffix variation. No model is used to create oracle labels.

3.3 Interfaces

The full-order rewrite interface asks the model to return:

```
{status, updated_order, clarification_question, reasons}
```

The current cart is shown with item positions for human-readable reference, but the output schema forbids positions and line IDs.

The line-ID patch interface asks the model to return:

```
{status, operations, clarification_question, reasons}
```

Each operation has an operation type, optional line ID, optional item, optional quantity and size updates, modifier add/remove lists, modifier reversal fields, and optional constraints. This is not a direct RFC 6902 schema; it is a domain-specific patch contract inspired by JSON Patch but designed to be auditable and executable against cart lines.

4 Evaluation Method

For every model, interface, and case, the runner sends one Nebius OpenAI-compatible chat-completion request at temperature zero with strict `json_schema` response formatting [7]. The evaluated models are Nebius-accessible open-weight models rather than a general leaderboard over all commercial systems. The raw result records provider success, response content, finish reason, usage, and latency.

The verifier first checks JSON extraction and interface schema validity. For rewrite outputs, it directly scores the returned `updated_order`. For patch outputs, it applies the operation list to the current cart, then strips line IDs and compares the resulting order with the oracle final order. The main metrics are:

- **Semantic success:** schema-valid output, correct status, exact final item multiset, exact constraints, executable patch operations, and fail-closed state preservation for non-accepted edits.
- **Unsafe state change:** accepting or changing state when the oracle requires clarification or safety rejection, or accepting a final order that conflicts with stated constraints.
- **Unintended drift:** changing a line that the oracle marks as unchanged.
- **Identity error:** a wrong-line error in a case with a target line.

4.1 Scoring Fairness

Both interfaces are reduced to the same object before semantic scoring: a status decision and a final cart state. For rewrite outputs, the final cart is the returned `updated_order`. For patch outputs, the verifier applies the returned operations to the current cart, strips line identifiers, and then compares the resulting cart to the same oracle used for rewrite scoring.

The verifier performs limited JSON extraction only: it accepts a raw JSON object, a fenced JSON object, or the substring from the first opening brace to the last closing brace. It does not repair missing fields, bad operation names, wrong primitive types, invented modifiers, or malformed item objects. Schema-invalid outputs receive `semantic_ok=false`, while component metrics still record any parsable partial behavior. Hallucinated cart lines, patches that reference absent line IDs, or stale-reference edits that mutate the cart are treated as execution failures. For non-accepted

oracle cases, a model must fail closed: the status must match the oracle and the requested cart edit must not be executed. This design intentionally favors neither interface after normalization; the patch interface exposes line IDs because stable line identity is the interface property under test.

For paired comparisons between rewrite and line-patch interfaces, the same case is evaluated under both interfaces for the same model. We report the patch-minus-rewrite rate difference, paired bootstrap 95% confidence intervals over cases, and an exact McNemar test on discordant paired outcomes.

5 Results

Table 1 gives the main results. All 5,040 provider calls completed. All evaluated models except Gemma-2-2B produced near-universal schema-valid outputs in both interfaces; Gemma-2-2B is retained as a low-end counterexample because line patch raised schema validity from 3.6% to 70.0% without producing exact semantic success.

Model	Interface	n	Provider	Schema	Semantic	Unsafe	Drift	Identity
Qwen3-235B-A22B	rewrite	360	100.0	100.0	86.1	1.9	0.0	0.0
Qwen3-235B-A22B	line patch	360	100.0	100.0	83.9	0.6	0.8	0.8
Qwen3-32B	rewrite	360	100.0	100.0	64.7	21.9	5.8	0.8
Qwen3-32B	line patch	360	100.0	100.0	80.8	7.2	4.4	0.0
Qwen3-30B-A3B	rewrite	360	100.0	100.0	74.2	6.9	3.6	0.0
Qwen3-30B-A3B	line patch	360	100.0	100.0	70.0	7.2	5.3	0.8
Llama-3.3-70B	rewrite	360	100.0	100.0	76.7	3.1	4.4	1.1
Llama-3.3-70B	line patch	360	100.0	100.0	72.8	2.8	2.8	0.0
Llama-3.1-8B	rewrite	360	100.0	100.0	25.3	9.7	29.4	15.8
Llama-3.1-8B	line patch	360	100.0	100.0	48.1	6.9	11.4	3.6
Gemma-2-2B	rewrite	360	100.0	3.6	0.0	1.9	0.8	0.0
Gemma-2-2B	line patch	360	100.0	70.0	0.0	11.1	0.0	0.0
Nemotron-Ultra-253B	rewrite	360	100.0	100.0	79.7	0.0	0.0	0.0
Nemotron-Ultra-253B	line patch	360	100.0	99.7	78.3	0.0	0.3	0.3

Table 1: Main results. Values other than n are percentages. “Semantic” is exact state-update success; “Unsafe” is an unsafe accepted or changed state; “Drift” is unintended change to protected cart lines.

5.1 Line IDs Help Where Identity Is the Bottleneck

The clearest positive effects appear for Qwen3-32B and Llama-3.1-8B. For Qwen3-32B, line patches improve semantic success from 64.7% to 80.8%, a +16.1 point paired difference with a 95% confidence interval of $[+11.1, +21.1]$ and McNemar $p < .001$. The same interface reduces unsafe state changes from 21.9% to 7.2%. For Llama-3.1-8B, patching improves semantic success from 25.3% to 48.1%, a +22.8 point difference with a 95% confidence interval of $[+16.9, +28.9]$ and McNemar $p < .001$. It also cuts unintended drift from 29.4% to 11.4%. These are the strongest evidence that stable line identifiers help when a model would otherwise rewrite unrelated cart state.

Table 2 shows that the gain is not universal. Qwen3-235B-A22B slightly favors rewrite on exact semantic success (86.1% versus 83.9%), Qwen3-30B-A3B also favors rewrite (74.2% versus 70.0%), Llama-3.3-70B favors rewrite (76.7% versus 72.8%), and Nemotron-Ultra-253B is nearly tied (79.7%

versus 78.3%). These differences are small relative to the large positive effects for Qwen3-32B and Llama-3.1-8B.

Model	Metric	Rewrite	Patch	Δ [95% CI]	p
Qwen3-235B-A22B	Semantic	86.1	83.9	-2.2 [-4.7, +0.3]	0.134
Qwen3-235B-A22B	Unsafe	1.9	0.6	-1.4 [-2.8, +0.0]	0.125
Qwen3-235B-A22B	Drift	0.0	0.8	+0.8 [+0.0, +1.9]	0.250
Qwen3-32B	Semantic	64.7	80.8	+16.1 [+11.1, +21.1]	<.001
Qwen3-32B	Unsafe	21.9	7.2	-14.7 [-18.6, -10.8]	<.001
Qwen3-32B	Drift	5.8	4.4	-1.4 [-3.3, +0.6]	0.267
Qwen3-30B-A3B	Semantic	74.2	70.0	-4.2 [-8.6, +0.3]	0.091
Qwen3-30B-A3B	Unsafe	6.9	7.2	+0.3 [-1.1, +1.7]	1.000
Qwen3-30B-A3B	Drift	3.6	5.3	+1.7 [+0.0, +3.3]	0.109
Llama-3.3-70B	Semantic	76.7	72.8	-3.9 [-7.8, +0.0]	0.076
Llama-3.3-70B	Unsafe	3.1	2.8	-0.3 [-0.8, +0.0]	1.000
Llama-3.3-70B	Drift	4.4	2.8	-1.7 [-3.1, -0.6]	0.031
Llama-3.1-8B	Semantic	25.3	48.1	+22.8 [+16.9, +28.9]	<.001
Llama-3.1-8B	Unsafe	9.7	6.9	-2.8 [-5.6, +0.0]	0.076
Llama-3.1-8B	Drift	29.4	11.4	-18.1 [-23.1, -13.3]	<.001
Gemma-2-2B	Semantic	0.0	0.0	+0.0 [+0.0, +0.0]	1.000
Gemma-2-2B	Unsafe	1.9	11.1	+9.2 [+6.1, +12.2]	<.001
Gemma-2-2B	Drift	0.8	0.0	-0.8 [-1.9, +0.0]	0.250
Nemotron-Ultra-253B	Semantic	79.7	78.3	-1.4 [-4.2, +1.4]	0.458
Nemotron-Ultra-253B	Unsafe	0.0	0.0	+0.0 [+0.0, +0.0]	1.000
Nemotron-Ultra-253B	Drift	0.0	0.3	+0.3 [+0.0, +0.8]	1.000

Table 2: Paired rewrite-versus-patch comparisons. Δ is line patch minus rewrite in percentage points.

5.2 Patch Interfaces Are Not Universally Better

Category-level results in Table 3 explain the mixed aggregate effect. Line patches help on duplicate-line removal (51.0% to 81.0%), quantity increases (73.8% to 81.9%), scoped add-ons (69.0% to 80.0%), and scoped size edits (64.3% to 74.3%). These are exactly the categories where stable line identity should reduce accidental whole-cart rewriting.

However, patching hurts modifier reversal (71.9% to 49.5%) and slightly hurts item replacement (51.4% to 48.1%). In modifier-reversal cases, the model must distinguish adding a modifier from removing an item from a previous **remove** list, or distinguish removing a previous add-on from adding a default removal. The patch interface makes the operation explicit, but also introduces operation-selection complexity. This is an important negative result: patch interfaces are a tool for identity and locality, not a general semantic reasoning solution.

Category	Interface	n	Semantic	Unsafe
add item	rewrite	210	66.7	5.7
add item	line patch	210	74.8	0.0
remove duplicate	rewrite	210	51.0	0.0
remove duplicate	line patch	210	81.0	0.0
qty increase	rewrite	210	73.8	0.5
qty increase	line patch	210	81.9	0.0
qty decrease	rewrite	210	77.1	0.0
qty decrease	line patch	210	80.0	0.0
scoped add-on	rewrite	210	69.0	4.3
scoped add-on	line patch	210	80.0	0.0
modifier reversal	rewrite	210	71.9	0.5
modifier reversal	line patch	210	49.5	0.0
scoped size	rewrite	210	64.3	2.9
scoped size	line patch	210	74.3	0.0
replace item	rewrite	210	51.4	11.0
replace item	line patch	210	48.1	0.0
safe constraint	rewrite	210	36.2	0.0
safe constraint	line patch	210	39.5	0.0
new conflict	rewrite	210	9.0	11.0
new conflict	line patch	210	0.0	14.8
unavailable edit	rewrite	210	51.0	32.9
unavailable edit	line patch	210	58.1	31.4
stale reference	rewrite	210	75.7	9.5
stale reference	line patch	210	76.7	15.2

Table 3: Semantic success and unsafe state-change rate by category, aggregated across the seven evaluated models.

5.3 Safety and Catalog Decisions Remain Hard

The hardest categories are safety and catalog boundary cases. Newly conflicting constraints have 9.0% semantic success under rewrite and 0.0% under line patch. Unavailable edits have high unsafe state-change rates: 32.9% under rewrite and 31.4% under line patch. Stale references also remain risky: semantic success is similar under rewrite and line patch (75.7% versus 76.7%), but unsafe state changes are higher under line patch (15.2% versus 9.5%).

This result matters because these failures occur after structured-output control. A schema can force the model to choose a valid status string and emit well-typed operations, but it cannot decide whether a newly stated peanut allergy invalidates existing satay noodles, whether mozzarella is unavailable for a vegan bowl, or whether the user is referring to a line that no longer exists.

5.4 Error Taxonomy

Table 4 separates the main failure types. Aggregated across models, line patch reduces unsafe state changes from 6.5% to 5.1%, drift from 6.3% to 3.6%, identity errors from 2.5% to 0.8%, status errors from 27.3% to 17.7%, order errors from 28.9% to 24.0%, and constraint errors from 25.7% to 16.7%. Operation errors remain a patch-specific failure mode, but they are lower than rewrite schema and extraction failures because most patch-capable models produce executable operations.

Interface	n	Unsafe	Drift	Identity	Status	Order	Constraint	Operation
line patch	2520	5.1	3.6	0.8	17.7	24.0	16.7	4.3
rewrite	2520	6.5	6.3	2.5	27.3	28.9	25.7	13.8

Table 4: Multi-label error taxonomy by interface. Values are percentages of cases. Labels do not sum to 100%.

6 Discussion

6.1 Interface Design Is a Reliability Control

The Qwen3-32B and Llama-3.1-8B results suggest a concrete engineering recommendation: if an LLM must edit persistent transactional state, the API should expose stable line IDs and prefer localized operations for identity-sensitive edits. This reduces the surface area over which the model can accidentally rewrite unrelated state. The result is consistent with the general motivation for patch formats and stable keys [2, 3], but the evidence here is transactional rather than purely document-editing oriented.

6.2 Patch APIs Need Domain Verifiers

The same results warn against treating patching as a complete guardrail. Patches improve locality, but unavailable edits and newly conflicting constraints are still semantic policy decisions. In a production ordering system, the LLM output should be treated as a candidate state transition. A deterministic verifier should check catalog membership, modifier availability, line existence, allergen and dietary constraints, and whether non-accepted edits leave state unchanged.

6.3 A Strong Model Narrows the Interface Gap

Qwen3-235B-A22B, Llama-3.3-70B, and Nemotron-Ultra-253B perform similarly under both interfaces. This does not make interface design irrelevant. It means that when the model is strong enough for a given edit distribution, line IDs mainly preserve good behavior rather than create large gains. For mid-sized or weaker models, the interface can materially change the error profile.

7 Threats to Validity

Synthetic domain. The benchmark uses a synthetic restaurant menu. This improves auditability and exact verification, but real ordering systems have larger catalogs, prices, inventory, tax, substitutions, user profiles, and payment state.

Single-edit setting. Each case evaluates one edit against a current cart with a short edit history rather than a fully interactive long conversation. This isolates state-update semantics, but does not measure dialogue recovery, memory across many turns, or user clarification behavior.

Benchmark size and templating. The v2 dataset has 360 deterministic cases. This is larger and more varied than the initial 120-case artifact, but it is still template-generated over a compact synthetic menu. A stronger version should add multiple menus, more catalog variation, paraphrases from real ordering logs where available, and interactive clarification turns.

Oracle strictness. Semantic success requires exact equality over item multisets and constraints. Some deviations may be recoverable in a real interface, but exactness is appropriate for a transaction contract that may be sent directly to an API.

Model and provider instability. The evaluated model IDs were available through Nebius at run time in May 2026. Provider routing, model snapshots, response-format support, and model behavior can change. The raw logs record exact model IDs and outputs to support replication.

Prompt and schema choices. The study compares two concrete interface designs, not every possible rewrite or patch prompt. A different patch operation vocabulary might improve modifier reversal while preserving the identity benefits observed here.

8 Conclusion

ORDERDELTABENCH shows that stateful transactional reliability is not solved by valid JSON. Stable line-ID patches can substantially reduce wrong-line updates and unintended cart drift, especially for weaker models, but they do not solve safety and catalog reasoning. The practical design pattern is therefore two-layered: expose state-local operation interfaces to reduce accidental rewrites, and enforce domain verification before any LLM-produced transaction is executed.

Data and Code Availability

The dataset, prompts, runner, verifier, raw model outputs, analysis scripts, and generated tables are included in the accompanying research artifact. The main raw result file is `results/raw/orderdelta_v2_expanded.jsonl`. The artifact does not store provider API keys; keys are read only from the execution environment.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit Author Statement

Yin Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work the author used OpenAI Codex/GPT-5 to assist with software implementation, experiment orchestration, literature organization, and manuscript drafting. The author reviewed and edited the outputs and takes full responsibility for the content of the manuscript and artifact.

Acknowledgements

No additional acknowledgements.

References

- [1] Sebastian Baltes, Florian Angermeir, Chetan Arora, et al. Guidelines for empirical studies in software engineering involving large language models, 2025.
- [2] Paul Bryan and Mark Nottingham. RFC 6902: Javascript object notation (JSON) patch. <https://www.rfc-editor.org/rfc/rfc6902.html>, 2013. Internet Engineering Task Force.
- [3] Sarel Duanis, Asnat Greenstein-Messica, and Eliya Habba. JSON Whisperer: Efficient JSON editing with LLMs, 2025.
- [4] JSON Schema Organization. JSON Schema Draft 2020-12. <https://json-schema.org/draft/2020-12>, 2022. Accessed 2026-05-16.
- [5] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024.
- [6] Shiva Krishna Reddy Malay, Shravan Nayak, Jishnu Sethumadhavan Nair, et al. EnterpriseOps-Gym: Environments and evaluations for stateful agentic planning and tool use in enterprise settings. <https://enterpriseops-gym.github.io/>, 2026. Accessed 2026-05-16.
- [7] Nebius. Nebius Token Factory API Documentation. <https://docs.tokenfactory.nebius.com/quickstart>, 2026. Accessed 2026-05-16.
- [8] OpenAI. Introducing structured outputs in the API. <https://openai.com/index/introducing-structured-outputs-in-the-api/>, 2024. Accessed 2026-05-16.
- [9] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.
- [10] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023.

- [11] Huaixiao Tou, Ying Zeng, Yuemeng Li, Cong Ma, Muzhi Li, Minghao Li, Weijie Yuan, He Zhang, and Kai Jia. ShoppingComp: Are LLMs really ready for your shopping cart?, 2025.
- [12] WebMall Authors. WebMall: A multi-shop benchmark for evaluating web agents. <https://wbsg-uni-mannheim.github.io/WebMall/>, 2026. Accessed 2026-05-16.
- [13] Jialin Yang, Dongfu Jiang, Lipeng He, et al. StructEval: Benchmarking LLMs’ capabilities to generate structural outputs, 2025.
- [14] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- [15] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, J. Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. SWE-bench goes live! In *Advances in Neural Information Processing Systems*, 2025.